客户端代码修改对比文档

本文档记录客户端代码（d:\project\WeChat）的所有修改内容，包含原代码与修改后代码的对比。

修改文件清单

序号	文件路径	修改内容
1	WeChat/ckernel.cpp	网络初始化移到子线程、修复信号连接错误、添加连接检查、修复 connect 警告
2	WeChat/logindialog.cpp	修复正则表达式 Bug、添加调试输出
3	WeChat/netapi/net/packdef.h	统一服务器 IP 宏
4	WeChat/usershow.cpp	H264 崩溃修复：slot_setImage 深拷贝、paintEvent 安全性增强
5	WeChat/VideoRender/h264encoder.cpp	H264 编码器标准化：av_frame_get_buffer、sws_scale
6	WeChat/vcc-client.pro	导入库格式修正：.lib → .dll.a
7	WeChat/ckernel.cpp（H264 部分）	编码器重初始化、Lambda 深拷贝

修改 1：ckernel.cpp

改动 ① 新增头文件

位置：文件开头第 7 行后

原代码：

```
#include "ckernel.h"
#include "QDebug"
#include "packdef.h"
#include "md5.h"
#include <QMessageBox>
```

```
#include <QTime>
#include <QBuffer>

#define NetPackMap(a) m_netPackMap[ a - DEF_PACK_BASE ]
```

修改后：

```
#include "ckernel.h"
#include "QDebug"
#include "packdef.h"
#include "md5.h"
#include <QMessageBox>
#include <QTime>
#include <QBuffer>
#include <thread>

#define NetPackMap(a) m_netPackMap[ a - DEF_PACK_BASE ]
```

说明：新增 <thread> 头文件，用于后续将网络初始化放到子线程执行。

改动 ② 构造函数中网络初始化部分

位置：构造函数中（约第 95~110 行）

原代码：

```
m_pClient = new TcpClientMediator;
m_pClient->OpenNet(m_serverIP.toStdString().c_str());
connect(m_pClient , SIGNAL(SIG_ReadyData(uint,char*,int)),this , SLOT(slot_dealData(uint,char*,int

//音频和视频的链接
for( int i = 0 ; i < 2 ; i++ )
{
    m_pAVClient[i] = new TcpClientMediator;
    m_pAVClient[i]->OpenNet( _DEF_SERVERIP , _DEF_PORT );
    connect(m_pClient , SIGNAL(SIG_ReadyData(uint,char*,int)),this , SLOT(slot_dealData(uint,char*
}
```

修改后：

```
m_pClient = new TcpClientMediator;
connect(m_pClient , SIGNAL(SIG_ReadyData(uint,char*,int)),this , SLOT(slot_dealData(uint,char*,int

//音频和视频的链接
for( int i = 0 ; i < 2 ; i++ )
{
```

```
    m_pAVClient[i] = new TcpClientMediator;
    //修复：原来错误地连接了 m_pClient，应该连接 m_pAVClient[i]
    connect(m_pAVClient[i] , SIGNAL(SIG_ReadyData(uint,char*,int)),this , SLOT(slot_dealData(uint,
}

//修复：网络初始化（阻塞式 connect）放到子线程执行，避免主线程阻塞导致 UI 无响应
std::thread([this](){
    m_pClient->OpenNet(m_serverIP.toStdString().c_str());
    for( int i = 0 ; i < 2 ; i++ )
    {
        m_pAVClient[i]->OpenNet( _DEF_SERVER_IP , _DEF_PORT );
    }
}).detach();
```

说明：修复了 3 个问题：

1. OpenNet 移入子线程，避免阻塞式 connect() 卡死 UI（原代码 3 次阻塞 connect 累计卡顿约 63 秒）
2. for 循环里 connect(m_pClient, ...) 改为 connect(m_pAVClient[i], ...)（原来音视频客户端的数据无法正确接收）
3. _DEF_SERVERIP 改为 _DEF_SERVER_IP，统一 IP

改动 ③ 构造函数中 RoomDialog 信号连接（修复 connect 警告）

位置：构造函数中（约第 85~91 行）

原代码：

```
connect(m_pRoomdialog , SIGNAL(SIG_close()) , this , SLOT(slot_quitRoom));
connect(m_pRoomdialog , SIGNAL(SIG_AudioPause()) , this , SLOT(slot_pauseAudio));
connect(m_pRoomdialog , SIGNAL(SIG_AudioStart()) , this , SLOT(slot_startAudio));
connect(m_pRoomdialog , SIGNAL(SIG_VideoPause()) , this , SLOT(slot_pauseVideo));
connect(m_pRoomdialog , SIGNAL(SIG_VideoStart()) , this , SLOT(slot_startVideo));
connect(m_pRoomdialog , SIGNAL(SIG_ScreenPause()) , this , SLOT(slot_pauseScreen));
connect(m_pRoomdialog , SIGNAL(SIG_ScreenStart()) , this , SLOT(slot_startScreen));
```

修改后：

```
connect(m_pRoomdialog , SIGNAL(SIG_close()) , this , SLOT(slot_quitRoom()));
connect(m_pRoomdialog , SIGNAL(SIG_AudioPause()) , this , SLOT(slot_pauseAudio()));
connect(m_pRoomdialog , SIGNAL(SIG_AudioStart()) , this , SLOT(slot_startAudio()));
connect(m_pRoomdialog , SIGNAL(SIG_VideoPause()) , this , SLOT(slot_pauseVideo()));
connect(m_pRoomdialog , SIGNAL(SIG_VideoStart()) , this , SLOT(slot_startVideo()));
connect(m_pRoomdialog , SIGNAL(SIG_ScreenPause()) , this , SLOT(slot_pauseScreen()));
connect(m_pRoomdialog , SIGNAL(SIG_ScreenStart()) , this , SLOT(slot_startScreen()));
```

说明：SLOT 宏中的槽函数名必须带括号 `()`，否则 Qt 会输出 "Parentheses expected" 警告，且信号槽连接失败，导致房间界面的音频/视频/桌面控制按钮无法工作。

改动 ④ slot_loginCommit 添加连接检查和调试输出

位置：`slot_loginCommit` 函数开头

原代码：

```
//提交登录信息
void ckernel::slot_loginCommit(QString tel, QString pass)
{
    std::string strTel = tel.toStdString();
    //std::string strPass = pass.toStdString();

    STRU_LOGIN_RQ rq;
    ...
```

修改后：

```
//提交登录信息
void ckernel::slot_loginCommit(QString tel, QString pass)
{
    qDebug() << "[DEBUG] slot_loginCommit 被调用, tel:" << tel;
    //修复：检查网络连接状态，未连接时提示用户，避免 SendData 内部阻塞重连导致 UI 卡死
    if( !m_pClient || !m_pClient->IsConnected() )
    {
        qDebug() << "[DEBUG] 网络未连接";
        QMessageBox::about(m_pLoginDlg, "提示", "网络未连接，请检查服务器是否启动后重试");
        return;
    }
    qDebug() << "[DEBUG] 网络已连接，准备发送登录请求";

    std::string strTel = tel.toStdString();
    //std::string strPass = pass.toStdString();

    STRU_LOGIN_RQ rq;
    ...
```

说明：

1. 添加连接状态检查，未连接时弹出提示框，避免 `SendData` 内部同步重连阻塞 UI 约 21 秒后静默失败
2. 添加 qDebug 调试输出，便于定位问题

改动 ⑤ slot_registerCommit 添加连接检查和调试输出

位置：slot_registerCommit 函数开头

原代码：

```
//发送注册信息
void ckernel::slot_registerCommit(QString tel, QString pass, QString name)
{
    std::string strTel = tel.toStdString();
    //std::string strPass = pass.toStdString();
    //中文
    std::string strName = name.toStdString();//格式utf-8

    STRU_REGISTER_RQ rq;
    ...
```

修改后：

```
//发送注册信息
void ckernel::slot_registerCommit(QString tel, QString pass, QString name)
{
    qDebug() << "[DEBUG] slot_registerCommit 被调用, tel:" << tel << " name:" << name;
    //修复：检查网络连接状态，未连接时提示用户，避免 SendData 内部阻塞重连导致 UI 卡死
    if( !m_pClient || !m_pClient->IsConnected() )
    {
        qDebug() << "[DEBUG] 网络未连接";
        QMessageBox::about(m_pLoginDlg, "提示", "网络未连接，请检查服务器是否启动后重试");
        return;
    }
    qDebug() << "[DEBUG] 网络已连接，准备发送注册请求";

    std::string strTel = tel.toStdString();
    //std::string strPass = pass.toStdString();
    //中文
    std::string strName = name.toStdString();//格式utf-8

    STRU_REGISTER_RQ rq;
    ...
```

说明：同改动 ④，添加连接检查和调试输出。

修改 2：logindialog.cpp

改动 ⑥ on_pb_commit_clicked 修复正则表达式 + 添加调试输出

位置：on_pb_commit_clicked 函数

原代码：

```
#include <QRegExp>
#include <QMessageBox>
//提交登录信息
void LoginDialog::on_pb_commit_clicked()
{
    QString strTel      = ui->le_tel->text();
    QString strPassword = ui->le_password->text();
    //校验
    //非空校验 清掉空格后，不能是空字符串
    QString tmpTel = strTel, tmpPass = strPassword;
    if(tmpTel.remove(" ").isEmpty() || tmpPass.remove(" ").isEmpty())
    {
        QMessageBox::about(this, "提示", "手机号密码不能为空格");
        return;
    }
    //1.校验手机号 正则表达式
    QRegExp reg("^1[3-8][0-9]\{6,9\}$");
    bool res = reg.exactMatch(strTel);
    if(!res)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
    }
    ...
```

修改后：

```
#include <QRegExp>
#include <QMessageBox>
//提交登录信息
void LoginDialog::on_pb_commit_clicked()
{
    qDebug() << "[DEBUG] on_pb_commit_clicked 被调用";
    QString strTel      = ui->le_tel->text();
    QString strPassword = ui->le_password->text();
    qDebug() << "[DEBUG] tel:" << strTel << " password length:" << strPassword.length();
    //校验
    //非空校验 清掉空格后，不能是空字符串
    QString tmpTel = strTel, tmpPass = strPassword;
    if(tmpTel.remove(" ").isEmpty() || tmpPass.remove(" ").isEmpty())
    {
        QMessageBox::about(this, "提示", "手机号密码不能为空格");
        return;
    }
    //1.校验手机号 正则表达式
    QRegExp reg("^1[3-8][0-9]{6,9}$");
```

```
    bool res = reg.exactMatch(strTel);
    qDebug() << "[DEBUG] 正则匹配结果:" << res;
    if(!res)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
    }
    ...
```

说明：

1. **关键 Bug 修复**：正则表达式 "\{6,9\}" 改为 "{6,9}"。在 C++ 字符串中，\{ 不是有效转义序列，编译器保留反斜杠，导致 QRegExp 收到的正则是 ^1[3-8][0-9]\{6,9\}$，其中 \{ 被当作字面量 {，**正则表达式永远不会匹配任何手机号**，导致校验总是失败。
2. 添加 qDebug 调试输出

改动 ⑦ on_pb_commit_register_clicked 修复正则表达式 + 添加调试输出

位置：on_pb_commit_register_clicked 函数

原代码：

```
//注册提交
void LoginDialog::on_pb_commit_register_clicked()
{

    QString strTel      = ui->le_tel_register->text();
    QString strPassword = ui->le_password_register->text();
    QString strConfirm  = ui->le_confirm_register->text();
    QString strName     = ui->le_name_register->text();
    //校验
    //非空校验 清掉空格后，不能是空字符串
    QString tmpTel = strTel, tmpPass = strPassword, tmpName = strName;
    if(tmpTel.remove(" ").isEmpty() || tmpPass.remove(" ").isEmpty() || tmpName.remove(" ").isEmpt
    {
        QMessageBox::about(this, "提示", "手机号密码昵称不能为空格");
        return;
    }
    //1.校验手机号 正则表达式
    QRegExp reg("^1[3-8][0-9]\{6,9\}$");
    bool res = reg.exactMatch(strTel);
    if(!res)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
```

```
    }
    ...
```

修改后：

```
//注册提交
void LoginDialog::on_pb_commit_register_clicked()
{
    qDebug() << "[DEBUG] on_pb_commit_register_clicked 被调用";
    QString strTel      = ui->le_tel_register->text();
    QString strPassword = ui->le_password_register->text();
    QString strConfirm  = ui->le_confirm_register->text();
    QString strName     = ui->le_name_register->text();
    qDebug() << "[DEBUG] tel:" << strTel << " name:" << strName << " pwd len:" << strPassword.leng
    //校验
    //非空校验 清掉空格后，不能是空字符串
    QString tmpTel = strTel, tmpPass = strPassword, tmpName = strName;
    if(tmpTel.remove(" ").isEmpty() || tmpPass.remove(" ").isEmpty() || tmpName.remove(" ").isEmpt
    {
        QMessageBox::about(this, "提示", "手机号密码昵称不能为空格");
        return;
    }
    //1.校验手机号 正则表达式
    QRegExp reg("^1[3-8][0-9]{6,9}$");
    bool res = reg.exactMatch(strTel);
    qDebug() << "[DEBUG] 正则匹配结果:" << res;
    if(!res)
    {
        QMessageBox::about(this, "提示", "手机号格式错误，8-11位手机号");
        return;
    }
    ...
```

说明：同改动 ⑥，修复正则表达式 Bug，添加调试输出。

修改 3：packdef.h

改动 ⑧ 统一服务器 IP 宏

位置：第 12 行

原代码：

```
#define _DEF_PORT   (8000)
#define _DEF_SERVERIP   ("192.0.2.1")
```

修改后：

```
#define _DEF_PORT   (8000)
#define _DEF_SERVERIP   ("192.0.2.129")
```

说明：与 INetMediator.h 中的 _DEF_SERVER_IP（值为 "192.0.2.129"）保持一致。原来两个宏指向不同 IP，导致 m_pClient 连 .129，m_pAVClient 连 .1，连到了不同地址。

修改 4：usershow.cpp（H264 崩溃修复 - 核心修复）

> 本部分为 H264 视频编码崩溃修复的核心修改。启用 H264 编码后，客户端在第一帧编码发送完成后立即崩溃，根因是 slot_setImage 中 QImage 浅拷贝（Qt 隐式共享）在多线程环境下导致竞态条件。详细分析参考 H264视频崩溃修复报告.md。

改动 ⑨ slot_setImage QImage 浅拷贝改为深拷贝

位置：UserShow::slot_setImage 函数

原代码：

```
void UserShow::slot_setImage(QImage &img)
{
    m_img = img;  // 浅拷贝：m_img 与 img 共享同一块像素数据
    m_lastTime = QTime::currentTime();
    this->update();
}
```

修改后：

```
void UserShow::slot_setImage(QImage &img)
{
    // [修复] 使用深拷贝而非浅拷贝，防止 paintEvent 中 scaled()
    //        访问已释放的共享像素数据
    // 原Bug: m_img = img 是浅拷贝（Qt隐式共享），m_img 与 img 共享同一块像素数据。
    //         slot_sendVideoFrame 返回后 img 被销毁，引用计数减少。
    //         paintEvent 中 m_img.scaled() 读取像素数据时，如果引用计数管理出现
    //         并发问题（video线程可能正在创建新的QImage影响共享数据），会导致读取
    //         已释放内存 → 崩溃。
    // 修复: m_img = img.copy() 创建深拷贝，m_img 拥有完全独立的像素数据缓冲区，
    //        不受其他 QImage 生命周期影响。
    m_img = img.copy();
    m_lastTime = QTime::currentTime();
```

```
    // [修复] 切换到 QImage 模式时隐藏 YuvGLWidget，防止 GL 渲染层覆盖
    if (m_glWidget) {
        m_glWidget->hide();
    }
    this->update();
}
```

说明：

1. **核心修复**：`m_img = img`（浅拷贝）改为 `m_img = img.copy()`（深拷贝）
2. **根因**：Qt 隐式共享在多线程环境下（Video线程创建QImage + Main线程paintEvent读取 + Worker线程delete[]堆操作），paintEvent中 `m_img.scaled()` 读取共享像素数据时发生竞态条件，访问已释放内存导致崩溃
3. 新增 `m_glWidget` 隐藏逻辑，防止 GL 渲染层覆盖 QImage 绘制

改动 ⑩ paintEvent 安全性增强

位置：`UserShow::paintEvent` 函数

原代码：

```
void UserShow::paintEvent(QPaintEvent *event)
{
    QPainter painter( this );
    painter.setBrush( QBrush( Qt::black ) );
    painter.drawRect( 0, 0, this->width(), this->height() );

    if( m_img.size().height()<=0 ) return;

    m_img = m_img.scaled( this->width() , this->height() - ui->lb_name->height(),
                          Qt::KeepAspectRatio );
    int x = this->width() - m_img.width();
    x = x/2;
    int y = this->height() - m_img.height() - ui->lb_name->height();
    y = ui->lb_name->height() + y/2;
    QPixmap pix = QPixmap::fromImage( m_img );
    painter.drawPixmap( x , y , pix );
    painter.end();
}
```

修改后：

```
void UserShow::paintEvent(QPaintEvent *event)
{
    QPainter painter( this );
    // 画黑色背景
    painter.setBrush( QBrush( Qt::black ) );
```

```
    painter.drawRect( 0, 0, this->width(), this->height() );

    // [修复] 当 YuvGLWidget 可见时（H264远程视频模式），跳过 QImage 绘制
    if (m_glWidget && m_glWidget->isVisible()) {
        painter.end();
        return;
    }

    if( m_img.size().height()<=0 ) return;

    // [修复] 添加 ui->lb_name 空指针检查和目标尺寸有效性检查
    int labelH = (ui->lb_name) ? ui->lb_name->height() : 20;
    int targetW = this->width();
    int targetH = this->height() - labelH;
    if (targetW <= 0 || targetH <= 0) {
        painter.end();
        return;
    }
    m_img = m_img.scaled(targetW, targetH, Qt::KeepAspectRatio);

    int x = this->width() - m_img.width();
    x = x/2;
    int y = this->height() - m_img.height() - labelH;
    y = labelH + y/2;
    QPixmap pix = QPixmap::fromImage( m_img );
    painter.drawPixmap( x , y , pix );
    painter.end();
}
```

说明：

1. 新增 `m_glWidget` 可见性检查，H264 远程视频模式下跳过 QImage 绘制
2. `ui->lb_name` 空指针检查：`(ui->lb_name) ? ui->lb_name->height() : 20`，防止 UI 文件变更后空指针崩溃
3. 目标尺寸有效性检查：`if (targetW <= 0 || targetH <= 0) return`，防止窗口最小化时 0 尺寸传入 `scaled()`
4. `labelH` 局部变量复用，减少重复调用

修改 5：h264encoder.cpp（H264 编码器标准化）

本部分为 H264 编码器 FFmpeg 帧缓冲区管理和色彩空间转换的标准化修复。

改动 ⑪ 帧缓冲区分配：av_image_alloc → av_frame_get_buffer

位置：`H264Encoder::init` 函数

原代码：

```
// 使用 av_image_alloc 手动分配
int bufSize = av_image_get_buffer_size(AV_PIX_FMT_YUV420P, width, height, 1);
m_yuvBuffer = (uint8_t*)av_malloc(bufSize);
av_image_fill_arrays(m_frame->data, m_frame->linesize, m_yuvBuffer,
                     AV_PIX_FMT_YUV420P, width, height, 1);

// RGB->YUV420P 转换器（当前未使用，保留接口）
m_swsCtx = nullptr;
```

修改后：

```
// 初始化时设置帧参数
m_frame->format = AV_PIX_FMT_YUV420P;
m_frame->width  = width;
m_frame->height = height;

// [修复] 使用 av_frame_get_buffer 替代 av_image_alloc
// 原因: av_image_alloc 绕过 AVBufferRef 引用计数机制，
//       avcodec_send_frame 内部 av_frame_ref 只能浅拷贝指针，
//       导致编码器与外部帧共享缓冲区。
int ret = av_frame_get_buffer(m_frame, 0);
if (ret < 0) {
    qCritical() << "H264Encoder: av_frame_get_buffer failed, ret=" << ret;
    freeResources();
    return false;
}

// 创建 RGB->YUV420P 转换器 (SwsContext)
m_swsCtx = sws_getContext(width, height, AV_PIX_FMT_RGB24,
                          width, height, AV_PIX_FMT_YUV420P,
                          SWS_BILINEAR, nullptr, nullptr, nullptr);
if (!m_swsCtx) {
    freeResources();
    return false;
}
```

说明：

1. av_image_alloc 绕过 AVBufferRef 引用计数机制，改为标准的 av_frame_get_buffer
2. 启用 sws_getContext 创建 RGB→YUV420P 转换器（原代码 m_swsCtx = nullptr 未使用）

改动 ⑫ RGB→YUV转换：手动 BT.601 → sws_scale

位置：H264Encoder::encode 函数

原代码：

```
// 手动 BT.601 RGB→YUV 转换
for (int y = 0; y < m_height; y++) {
    for (int x = 0; x < m_width; x++) {
        uint8_t r = row[x * 3];
        // ... 手动计算 Y/U/V 分量
        yPlane[y * yStride + x] = yVal;
        if ((y & 1) == 0 && (x & 1) == 0) {
            uPlane[(y >> 1) * uStride + (x >> 1)] = uVal;
            vPlane[(y >> 1) * vStride + (x >> 1)] = vVal;
        }
    }
}
```

修改后：

```
// 确保帧缓冲区可写
int ret = av_frame_make_writable(m_frame);
if (ret < 0) return QByteArray();

// [修复] 使用 FFmpeg sws_scale 替代手动 BT.601 转换
// 前提: FFmpeg DLLs 是 MinGW 编译的（确认依赖 msvcrt.dll），
//       配合 .dll.a 导入库可以安全使用
const uint8_t *srcData[1] = { rgbImg.constBits() };
int srcLinesize[1] = { rgbImg.bytesPerLine() };
sws_scale(m_swsCtx, srcData, srcLinesize, 0, m_height,
          m_frame->data, m_frame->linesize);
```

说明：

1. 手动 BT.601 转换改为 sws_scale，SIMD 优化、边界检查完善
2. 新增 av_frame_make_writable 确保帧缓冲区可写（引用计数 > 1 时自动分配新缓冲区）
3. **重要前提**：已验证 FFmpeg DLLs 是 MinGW 编译的（依赖 msvcrt.dll，非 msvcrXXX.dll），sws_scale 可以安全使用

修改 6：vcc-client.pro（导入库格式修正）

改动 ⑬ FFmpeg 导入库 .lib → .dll.a

位置：vcc-client.pro LIBS 配置

原代码：

```
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.lib \
        $$PWD/ffmpeg-4.2.2/lib/libavformat.lib \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.lib \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.lib
```

修改后：

```
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavdevice.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavfilter.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavformat.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libpostproc.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswresample.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.dll.a
```

说明：

1. MSVC 格式 `.lib` 改为 MinGW 标准格式 `.dll.a`，符号修饰完全匹配
2. 补全所有 FFmpeg 库引用（原代码仅引用 4 个，修改后引用全部 8 个）
3. `videorender.pri` 中的 FFmpeg 库引用也需同步修改为 `.dll.a` 格式

导入库格式	编译器	说明
`.lib`	MSVC	COFF 格式，符号修饰与 MinGW 不完全匹配
`.dll.a`	MinGW	标准 GNU 格式，符号修饰完全匹配

修改 7：ckernel.cpp（H264 编码相关）

改动 ⑭ 编码器重初始化：close()+init() → delete+new

位置：`ckernel::slot_startVideo` 函数

原代码：

```
m_h264Encoder->close();
bool initOk = m_h264Encoder->init(encWidth, encHeight, encFps, encBitrate);
```

修改后：

```
// [修复] 销毁旧实例，创建全新实例
// 原代码 close()+init() 在同一对象上重初始化，
// FFmpeg 全局状态可能被污染，新的 SwsContext 在 sws_scale 时可能访问损坏内存
delete m_h264Encoder;
m_h264Encoder = new H264Encoder(this);
bool initOk = m_h264Encoder->init(encWidth, encHeight, encFps, encBitrate);
```

说明：close() 释放 codecCtx 后，FFmpeg 全局状态可能被污染。在同一对象上调用 init() 创建的新 SwsContext 在 sws_scale 时可能访问损坏内存。使用 delete + new 创建全新实例，确保无残留状态。

改动 ⑮ Lambda 捕获 QImage 深拷贝

位置：ckernel::slot_sendVideoFrame 函数

原代码：

```
QTimer::singleShot(0, this, [this, img]() {
    encodeAndSendVideoFrame(img);
});
```

修改后：

```
// [修复] 深拷贝捕获，独立像素数据
// 原代码 [this, img] 捕获 img 是浅拷贝（Qt 隐式共享）
// lambda 被投递到事件队列后，摄像头线程可能在 lambda 执行前
// 释放或修改 img 的底层像素数据，导致 encode() 中访问已释放内存
QImage imgCopy = img.copy();  // 深拷贝，独立像素数据
QTimer::singleShot(0, this, [this, imgCopy]() {
    encodeAndSendVideoFrame(imgCopy);
    m_encodingPending = false;
});
```

说明：[this, img] 捕获 img 是浅拷贝（Qt 隐式共享）。lambda 被投递到事件队列后，摄像头线程可能在 lambda 执行前释放或修改 img 的底层像素数据，导致 encode() 中访问已释放内存。改用 img.copy() 深拷贝捕获。

修改效果总结

问题	修复前	修复后
UI 冻结	构造函数 3 次阻塞 connect，卡 ~63 秒	网络初始化在子线程，UI 立即可交互

问题	修复前	修复后
信号连接错误	AV 客户端数据发到错误的 m_pClient	正确连接到 m_pAVClient[i]
点击提交无反应	SendData 静默阻塞重连，无提示	弹出"网络未连接"提示框
IP 不一致	m_pClient 连 .129，m_pAVClient 连 .1	全部统一为 .129
正则表达式永远不匹配	`\{6,9\}` 导致校验总是失败	`{6,9}` 正确匹配手机号
RoomDialog 信号槽连接失败	SLOT 缺少括号，连接失败	添加括号，连接成功
H264 第一帧后崩溃	**QImage 浅拷贝导致多线程竞态**	**m_img = img.copy() 深拷贝**
FFmpeg 帧缓冲区共享	**av_image_alloc 绕过引用计数**	**av_frame_get_buffer + av_frame_make_writable**
RGB→YUV 转换越界风险	**手动 BT.601 转换**	**sws_scale 标准函数**
导入库符号修饰不匹配	**.lib (MSVC 格式)**	**.dll.a (MinGW 格式)**
编码器重初始化状态残留	**close()+init()**	**delete+new 全新实例**
Lambda 捕获 use-after-free	**[img] 浅拷贝捕获**	**img.copy() 深拷贝捕获**

重要提醒

1. **服务器必须启动**：客户端代码已修复，但需要服务器正常运行才能完成注册/登录流程。
2. **服务器 IP 配置**：如果服务器 IP 不是 `192.0.2.129`，请同步修改：
   - `packdef.h` 的 `_DEF_SERVERIP`
   - `INetMediator.h` 的 `_DEF_SERVER_IP`
   - 或在程序运行目录下创建 `config.ini` 配置文件指定正确的 IP
3. **config.ini 格式示例**：

```
[Net]
```

```
ip=192.0.2.129
```

4. **验证服务器连通性**（PowerShell）：

```
Test-NetConnection -ComputerName 192.0.2.129 -Port 8000
```